COMSATS University Islamabad

Vehari Campus

Department of Computer Science

Final Year Project (FYP)

Evaluation III | 60% Completion

AI Mock Interviewer

Ghost Interview

Submitted by:

Ehsan Hamza (SP23-BCS-019)

Shaharyar Altaf (SP23-BCS-055)

Supervisor:

Sir Qizar Javed

Evaluation Date: 13 October 2026

This document covers Implementation (Chapter 5) and Software Testing (Chapter 6).

Table of Contents

1. Introduction

1.1 Project Objectives

1.2 Technology Stack

2. Implementation (60% Modules Completion)

2.1 User Registration Module

2.2 User Login & Authentication Module

2.3 Job Role Selection & Interview Initiation Module

2.4 AI Interview Session Module

2.5 Speech-to-Text, Filler-Word & Sentiment Analysis Module

2.6 Facial Emotion Detection Module

2.7 Confidence & Competency Report Module

2.8 Interview History Dashboard Module

2.9 Admin Panel Module

2.10 Backend REST API & Security Module

2.11 Module Completion Summary

3. Software Testing

3.1 Authentication Module Testing

3.2 Interview Workflow & AI Module Testing

3.3 Reporting, Dashboard & API Testing

4. Conclusion

1. Introduction

Ghost Interview is an AI-powered mock-interview platform that gives job seekers a realistic, private environment in which to practise. A candidate selects a job role, then sits a live interview conducted by an AI avatar that asks each question aloud, captures the candidate's spoken answers through the microphone, and watches facial expressions through the webcam. When the session ends, the system analyses speech quality, filler-word usage, sentiment and non-verbal behaviour, and produces a Confidence and Competency Report with an overall readiness score, strengths and improvement areas that the candidate can export as a PDF.

This document reports the state of the project at Evaluation III. It presents the implemented modules together with their algorithms and screenshots captured from the running application (Chapter 5 / Implementation), and the software testing performed on those modules, including both positive and negative scenarios (Chapter 6 / Software Testing). The features described here represent approximately sixty percent of the overall project scope.

1.1 Project Objectives

- Provide a realistic AI-driven mock interview where an avatar speaks each question and the candidate answers by voice.
- Authenticate candidates and administrators securely with role-based access and hardened credential handling.
- Convert spoken answers to text and analyse filler words and sentiment to assess communication quality.
- Detect facial expressions in the browser during the interview to measure confidence, eye contact and nervousness.
- Generate a Confidence and Competency Report with a readiness score, strengths, improvement areas and one-click PDF export.
- Track improvement over time through an interview-history dashboard and platform-wide admin analytics.

1.2 Technology Stack

Layer	Technology	Purpose
Frontend	React 19 + Vite	Single-page application UI and routing.
Styling	Tailwind CSS 4	Responsive, utility-first design system.
Backend	Node.js 20 + Express	REST API, session orchestration, business logic.
Database	MongoDB Atlas (Mongoose)	Persistent storage for users, sessions and reports.
Authentication	JWT + refresh cookie	Short-lived access token in memory; httpOnly refresh cookie.
AI — Questions	Google Gemini API	Interview question generation with curated fallback.
AI — Speech	OpenAI Whisper + Web Speech API	Speech-to-text with browser-native fallback.
AI — Voice	ElevenLabs API	AI avatar voice synthesis with browser TTS fallback.

Layer	Technology	Purpose
AI — Emotion	face-api.js (browser)	In-browser facial expression detection; no video uploaded.
Analysis	sentiment (AFINN)	Lexicon-based sentiment scoring of transcripts.

2. Implementation (60% Modules Completion)

This chapter documents the modules that have been implemented and verified working at Evaluation III. For each module the functionality is described, the underlying algorithm is presented in a step-by-step algorithm table following the thesis template format, and the working application is demonstrated through screenshots captured from the live system.

2.1 User Registration Module

Functionality: A new candidate creates an account by providing a full name, a valid e-mail address and a password. The system validates each field, rejects duplicate e-mail addresses with a constant response to prevent account enumeration, hashes the password with bcrypt (cost factor 12), issues a JWT session, and dispatches a welcome e-mail (logged to the console in development when SMTP is not configured).

Algorithm 2.1: User Registration

Step	Description
1	User fills the registration form with full name, e-mail and password.
2	Validate input: name must be 2–100 characters; e-mail must match a valid e-mail format; password must meet the minimum strength requirement.
3	If any validation fails, return HTTP 400 with field-level messages and abort.
4	Query the users collection for an existing record with the same e-mail address.
5	If an account with that e-mail already exists, return HTTP 409 "An account with this email already exists" and abort.
6	Hash the password with bcrypt (cost factor 12) and create the user document with role defaulting to candidate and status active.
7	Send the welcome e-mail asynchronously (or log it when SMTP is not configured).
8	Return HTTP 201 with the safe user object (password hash and tokens excluded).

Screenshots



Figure 2.1: Registration form with name, e-mail and password fields.



Figure 2.2: Validation errors for invalid or missing input (HTTP 400).



Figure 2.3: Duplicate e-mail rejected with an HTTP 409 conflict message.



Figure 2.4: Successful registration — account created and safe user returned.

2.2 User Login & Authentication Module

Functionality: A registered user signs in with e-mail and password. The server verifies the credentials against the stored bcrypt hash, enforces brute-force lockout after five failed attempts, blocks suspended accounts, and issues a short-lived JWT access token (kept in memory on the client) together with a long-lived refresh token in an httpOnly, Secure, SameSite=strict cookie. A refresh endpoint rotates both tokens, and logout-all invalidates every outstanding session by bumping the token version.

Algorithm 2.2: User Login & Authentication

Step	Description
1	User enters e-mail and password on the login form and submits.
2	Validate input: e-mail must be valid; password must not be empty. On failure return HTTP 400.
3	Fetch the user by e-mail, explicitly selecting the hidden passwordHash field.
4	If no active user is found, return HTTP 401 "Invalid email or password" (constant response to avoid user enumeration).
5	If the account is locked from repeated failures, return HTTP 429; if suspended, return HTTP 403.
6	Compare the submitted password with the stored bcrypt hash using a constant-time comparison.
7	On mismatch, increment failedLoginAttempts; after five failures set lockUntil for 15 minutes; return HTTP 401.
8	On success, reset the failed-attempt counter, update lastLoginAt, and save.
9	Issue a JWT access token and a refresh token stored in an httpOnly cookie; return HTTP 200 with the token and safe user.
10	The client keeps the access token in memory and redirects the user to the dashboard.

Screenshots



Figure 2.5: Login interface with e-mail and password fields.



Figure 2.6: Successful login with redirection to the dashboard.



Figure 2.7: Error message shown for an incorrect password (HTTP 401).



Figure 2.8: Required-field validation when the form is submitted empty.



Figure 2.9: Account-lockout message after repeated failed logins (HTTP 429).

2.3 Job Role Selection & Interview Initiation Module

Functionality: The candidate browses the active job roles and selects one to begin an interview. The server snapshots the questions onto the session — generating fresh questions via the Gemini API when configured, otherwise falling back to the role's curated question bank — so that later edits to a role never change an in-progress or completed interview. A new InterviewSession is created with status active and the first question time-stamped.

Algorithm 2.3: Job Role Selection & Interview Initiation

Step	Description
1	Candidate opens the job-role selection screen; the client requests the active roles (GET /api/job-roles).
2	Validate the active job role exists and has at least one configured question; otherwise return HTTP 404/400.
3	Attempt to generate interview questions via the Gemini API for the selected role.
4	If Gemini is unavailable or returns too few questions, fall back to the role's curated question bank ordered by their sequence.
5	Snapshot the resolved question texts onto the session so later role edits cannot alter this interview.
6	Create the InterviewSession with status active, currentQuestionIndex 0, and the first question's askedAt time-stamp.
7	Return HTTP 201 with the session id, total question count and the first question.

Screenshots



Figure 2.10: Job-role selection screen with available roles.



Figure 2.11: Interview initiated — first question presented to the candidate.

2.4 AI Interview Session Module

Functionality: During a live interview the AI avatar speaks each question (ElevenLabs voice when configured, otherwise the browser's speech synthesis), the candidate answers by voice with live speech-to-text (OpenAI Whisper when configured, otherwise the browser Web Speech API), and the webcam runs in-browser emotion detection throughout. Only microphone audio is uploaded when Whisper is configured; raw webcam video never leaves the browser.

Algorithm 2.4: AI Interview Session

Step	Description
1	Candidate starts the session; the client requests camera and microphone permission.
2	For the current question the client requests the spoken audio (GET /api/sessions/:id/voice); the server synthesises it with ElevenLabs, else the browser speaks it.
3	The candidate answers by voice; the browser captures audio and produces a live transcript (Web Speech API, or Whisper via POST /api/sessions/:id/transcribe when configured).
4	The client submits the transcript (POST /api/sessions/:id/responses).
5	Meanwhile the webcam stream is analysed in-browser with face-api.js and emotion readings are batched to POST /api/sessions/:id/emotion.
6	The server stores the SpeechRecord, advances to the next question, or sets status to processing on the last question.
7	When all questions are answered the candidate completes the session to trigger report generation.

Screenshots

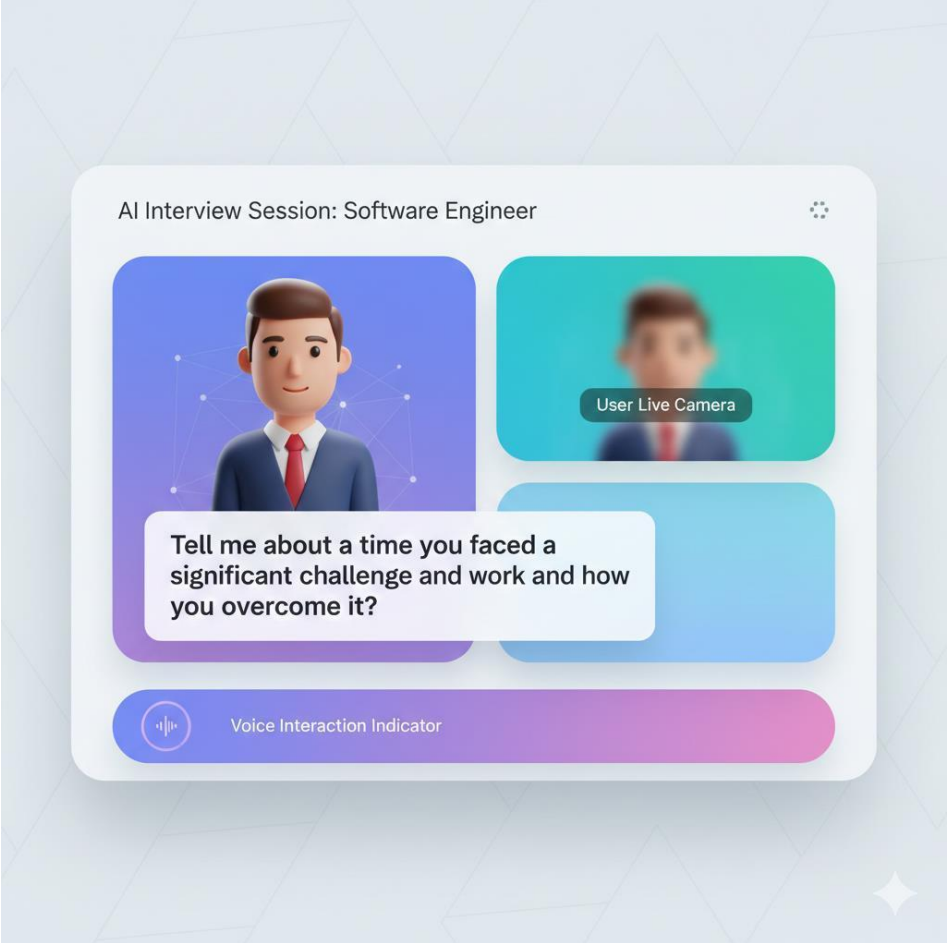


Figure 2.12: Live interview screen with AI avatar, question text and webcam feed.

Figure 2.13: Live speech-to-text transcript of the candidate's spoken answer.



Figure 2.14: Browser-side emotion detection running over the webcam feed.

2.5 Speech-to-Text, Filler-Word & Sentiment Analysis Module

Functionality: Each submitted transcript is analysed in two deterministic passes. A word-boundary regular expression counts filler words and phrases (um, uh, like, you know, and so on), and the AFINN sentiment lexicon scores the emotional tone, clamped to a −1..1 range and labelled positive, negative or neutral. The results are stored on the SpeechRecord for later aggregation in the report.

Algorithm 2.5: Speech, Filler-Word & Sentiment Analysis

Step	Description
1	Receive the transcribed answer for the current question.
2	Lower-case the transcript and match it against a filler-word/phrase regex with word boundaries.
3	Count the total filler-word occurrences and record the distinct filler words used.
4	Run the AFINN sentiment analyzer to obtain a comparative score for the transcript.
5	Clamp the score to the −1..1 range and label it positive (> 0.1), negative (< −0.1) or neutral.
6	Persist a SpeechRecord with the transcript, fillerWordCount, fillerWords, sentimentScore and sentimentLabel.
7	Return the per-answer analysis to the client for immediate feedback.

Screenshots



Figure 2.15: Per-answer analysis showing filler-word count and sentiment.



Figure 2.16: Filler words detected across a full interview session.

2.6 Facial Emotion Detection Module

Functionality: Facial expressions are analysed entirely in the browser using the pre-trained face-api.js model. For each sampled frame the model detects the dominant expression and derives eye-contact and nervousness indicators; these numeric readings — never the raw video — are batched and sent to the server, which stores them as EmotionRecord documents for the session.

Algorithm 2.6: Facial Emotion Detection

Step	Description
1	Load the pre-trained face-api.js models in the browser at session start.
2	Start a detection loop over the webcam stream, sampling frames at a fixed interval.
3	For each detected face, take the dominant expression as the emotion label.
4	Derive eye-contact and nervousness indicators from the detection results.
5	Batch the readings captured for the current question and POST them to /api/sessions/:id/emotion.
6	Validate the batch and bulk-insert the EmotionRecord documents linked to the session.
7	No raw webcam video is uploaded; only the derived numeric readings are stored.

Screenshots



Figure 2.17: Real-time facial expression detection during the interview.



Figure 2.18: Batched emotion readings recorded for the session.

2.7 Confidence & Competency Report Module

Functionality: When a session is completed (or terminated early) the server loads all SpeechRecords and EmotionRecords and computes a deterministic Confidence and Competency Report: communication, confidence, emotional-stability and competency scores plus an overall readiness score out of 10. It also builds strengths and improvement areas and an emotion summary with a distribution and timeline. The report screen renders the scores and offers a one-click PDF export generated in the browser.

Algorithm 2.7: Confidence & Competency Report Generation

Step	Description
1	On complete/terminate, load the session's SpeechRecords (ordered) and EmotionRecords.
2	Compute answered ratio = answered questions / total questions.
3	Aggregate filler words across answers and compute fillerRatio = total filler words / total words.
4	Compute the positive-sentiment ratio and average sentiment across answers.
5	Build the emotion summary: dominant emotion, distribution percentages, average eye contact and nervousness, and a down-sampled timeline.
6	Derive communication = 100 − fillerRatio×400; confidence from eye contact, calmness and positive tone; emotional stability = (1 − nervousness)×100; competency as a weighted blend.
7	Compute overall readiness = average of confidence, competency and emotional stability, scaled to 0–10.
8	Assemble strengths and improvement areas from threshold rules on the computed metrics.
9	Upsert the Report, set the session status to completed, and return it; the client renders the scores and can export a PDF.

Screenshots

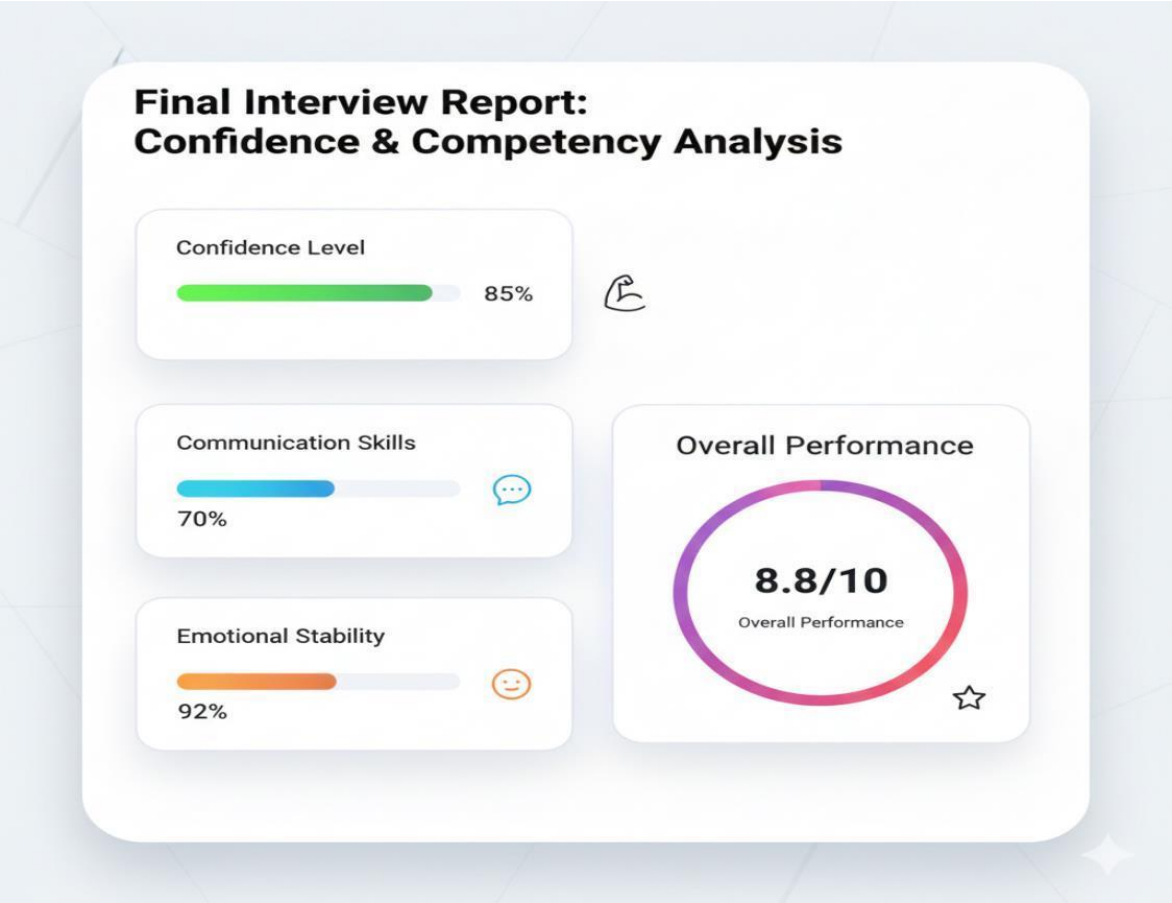


Figure 2.19: Confidence & Competency Report with overall readiness ring.



Figure 2.20: Performance breakdown — confidence, competency, communication and stability.



Figure 2.21: Emotion distribution and strengths / improvement areas.



Figure 2.22: One-click PDF export of the report.

2.8 Interview History Dashboard Module

Functionality: The dashboard shows the candidate their interview history and past reports. Sessions and reports are returned paginated and sorted newest-first, scoped strictly to the authenticated user, with each report summarised by role, date and readiness so the candidate can track improvement over time.

Algorithm 2.8: Interview History Dashboard

Step	Description
1	Candidate opens the dashboard; the client requests sessions (GET /api/sessions) and reports (GET /api/reports).
2	Scope both queries to the authenticated user and apply pagination and newest-first sorting.
3	Return each session with its role, status, progress and times, and each report with its scores and generation date.
4	Render history cards linking to the full report for each completed interview.

Screenshots



Figure 2.23: Dashboard showing interview history and past reports.



Figure 2.24: Performance trends across multiple sessions.

2.9 Admin Panel Module

Functionality: Administrators manage users and job roles and view platform-wide analytics. User management supports search, role/status filters, and suspend/reactivate or soft-delete, with guards preventing admins from affecting themselves or other admins and force-logging-out suspended users by bumping their token version. Job-role management supports create, edit and deactivate (soft delete). The analytics overview aggregates counts and average confidence, competency and readiness.

Algorithm 2.9: Admin Panel

Step	Description
1	Admin opens the admin panel; every admin route enforces role === 'admin'.
2	User management lists accounts with search, role and status filters; admins can suspend, reactivate or soft-delete candidates.
3	Reject attempts to suspend/delete other admins or the current admin; bump tokenVersion on suspend/delete to force logout.
4	Job-role management lets admins create, edit and deactivate roles and their question banks (soft delete preserves history).
5	The analytics endpoint aggregates platform-wide counts and average confidence, competency and readiness scores.

Screenshots



Figure 2.25: Admin user-management panel with search and status controls.



Figure 2.26: Admin job-role management screen.



Figure 2.27: Platform-wide analytics overview.

2.10 Backend REST API & Security Module

Functionality: The Express backend exposes a documented REST API grouped under /auth, /job-roles, /sessions, /reports and /admin, with a /health probe. Every request is hardened with Helmet security headers, request logging, Zod input validation, express-mongo-sanitize against NoSQL injection, rate limiting (tighter on auth routes), CSRF-guarded refresh, JWT authentication and per-resource ownership checks.

Screenshots



Figure 2.28: API health check (GET /api/health) returning status ok.



Figure 2.29: Authenticated API response with JWT and safe user payload.



Figure 2.30: Security response for an unauthenticated or CSRF-missing request.

2.11 Module Completion Summary

Module	Status	Key Capability

Module	Status	Key Capability
User Registration	Completed	Validation, duplicate check, bcrypt hashing, welcome e-mail.
User Login & Authentication	Completed	Credential verification, brute-force lockout, JWT + refresh tokens.
Job Role Selection & Initiation	Completed	Active-role listing, Gemini/curated question snapshot.
AI Interview Session	Completed	AI avatar voice, live speech-to-text, webcam emotion capture.
Speech, Filler-Word & Sentiment	Completed	Filler-word detection and AFINN sentiment per answer.
Facial Emotion Detection	Completed	In-browser face-api.js detection; only readings stored.
Confidence & Competency Report	Completed	Deterministic scoring, PDF export, strengths/improvements.
Interview History Dashboard	Completed	Per-user paginated session and report history.
Admin Panel	Completed	User & job-role management, platform analytics.
Backend REST API & Security	Completed	Helmet, Zod, sanitization, rate limiting, RBAC, ownership.
Hume / DeepFace Server Integration	In Progress	Specified provider not yet integrated; browser model used.
Production Deployment Validation	In Progress	HTTPS deploy, load and real-device testing pending.

3. Software Testing

This chapter documents the software testing carried out on the completed modules. Each module is covered by test cases that specify a Test Case ID, scenario, input data, expected result, actual result and a Pass/Fail status. Both positive (valid) and negative (invalid) scenarios are included, and screenshots captured from the live system provide evidence of the actual results.

3.1 Authentication Module Testing

The registration and login flows were tested with valid credentials, an incorrect password, empty fields, an unregistered e-mail, a duplicate e-mail, and repeated failed logins to confirm the brute-force lockout.

TC ID	Scenario	Input Data	Expected Result	Actual Result	Status
TC-L1	Login with valid credentials	candidate@ghost.demo / Password123!	Successful login, JWT issued, redirect to dashboard	Logged in, redirected to dashboard	**PASS**
TC-L2	Login with incorrect password	candidate@ghost.demo / WrongPass1	Error: Invalid email or password	Error banner shown, no token issued	**PASS**
TC-L3	Login with empty fields	(submit empty form)	Required-field validation	Validation messages displayed	**PASS**
TC-L4	Login with unregistered e-mail	ghost@ghost.demo / Password123!	Invalid email or password	Authentication failure shown	**PASS**
TC-L5	Repeated failed logins	5 wrong-password attempts	Account locked (HTTP 429) after 5 tries	Lockout message shown	**PASS**
TC-R1	Register with valid information	New name, unique e-mail, strong password	Account created, welcome e-mail, success shown	Account created, logged in	**PASS**
TC-R2	Register with existing e-mail	candidate@ghost.demo (already exists)	Duplicate e-mail error (409)	"An account with this email already exists"	**PASS**
TC-R3	Register with invalid input	Bad e-mail format, short password	Field-level validation errors (400)	Validation messages displayed	**PASS**

Screenshots



Figure 3.1: TC-L1 — successful login and redirection.



Figure 3.2: TC-L2 — error for incorrect password.



Figure 3.3: TC-L3 — required-field validation for empty fields.



Figure 3.4: TC-L5 — account lockout after repeated failed logins.



Figure 3.5: TC-R2 — duplicate e-mail error.

3.2 Interview Workflow & AI Module Testing

The interview workflow was tested end-to-end: starting a session, submitting answers, batching emotion readings, and completing the session. Speech and sentiment analysis, the AI providers' fallback behaviour, and permission handling were verified against the live system.

TC ID	Scenario	Input Data	Expected Result	Actual Result	Status
TC-I1	Start interview session	Select active job role	Session created, first question shown	Session active, question displayed	**PASS**
TC-I2	Submit answer & advance	Spoken answer for Q1	Analysis returned, next question shown	Filler/sentiment analysis + next question	**PASS**
TC-I3	Record emotion batch	face-api.js readings for Q1	Readings stored for session	"Recorded N emotion reading(s)"	**PASS**
TC-I4	Complete session & report	All questions answered	Confidence & competency report generated	Report with scores and insights shown	**PASS**
TC-I5	Deny camera/mic permission	Block browser permissions	Graceful permission error, session halts	Permission error shown, no crash	**PASS**
TC-S1	Filler-word detection	Transcript with um / uh / like	Filler words counted	Correct count and list returned	**PASS**
TC-S2	Sentiment scoring	Positive and negative transcripts	Correct positive/neutral/negative label	Labels match transcript tone	**PASS**

Screenshots



Figure 3.6: TC-I1 — interview session started with first question.



Figure 3.7: TC-I2 — answer submitted with per-answer analysis.



Figure 3.8: TC-I3 — emotion batch recorded during the session.



Figure 3.9: TC-I4 — report generated after completing the session.

3.3 Reporting, Dashboard & API Testing

The remaining completed modules were tested against the live system to confirm correct data retrieval, rendering, ownership enforcement and API responses, including PDF export and admin analytics.

TC ID	Scenario	Input Data	Expected Result	Actual Result	Status
TC-P1	View report & export PDF	Completed session id	Report rendered, PDF downloaded	Report shown, PDF downloaded	**PASS**
TC-P2	Partial report on early exit	Terminate after 1 answer	Partial report with isPartial = true	Partial report shown with notice	**PASS**
TC-D1	List interview history	GET /api/sessions & /api/reports	Paginated history returned	History dashboard populated	**PASS**
TC-O1	Ownership check on report	Another user's session id	Forbidden (403)	"This report does not belong to you"	**PASS**
TC-H1	API health check	GET /api/health	status = ok	{"success":true,...} returned	**PASS**
TC-A1	Admin analytics overview	GET /api/admin/analytics (admin)	Platform counts + average scores	Analytics dashboard populated	**PASS**
TC-A2	RBAC blocks non-admin	GET /api/admin/users (candidate token)	Forbidden (403)	Access denied	**PASS**

Screenshots



Figure 3.10: TC-P1 — Confidence & Competency Report with PDF export.



Figure 3.11: TC-P2 — partial report notice on early termination.



Figure 3.12: TC-D1 — interview history dashboard.



Figure 3.13: TC-H1 — API health check response.



Figure 3.14: TC-A1 — admin analytics overview.

4. Conclusion

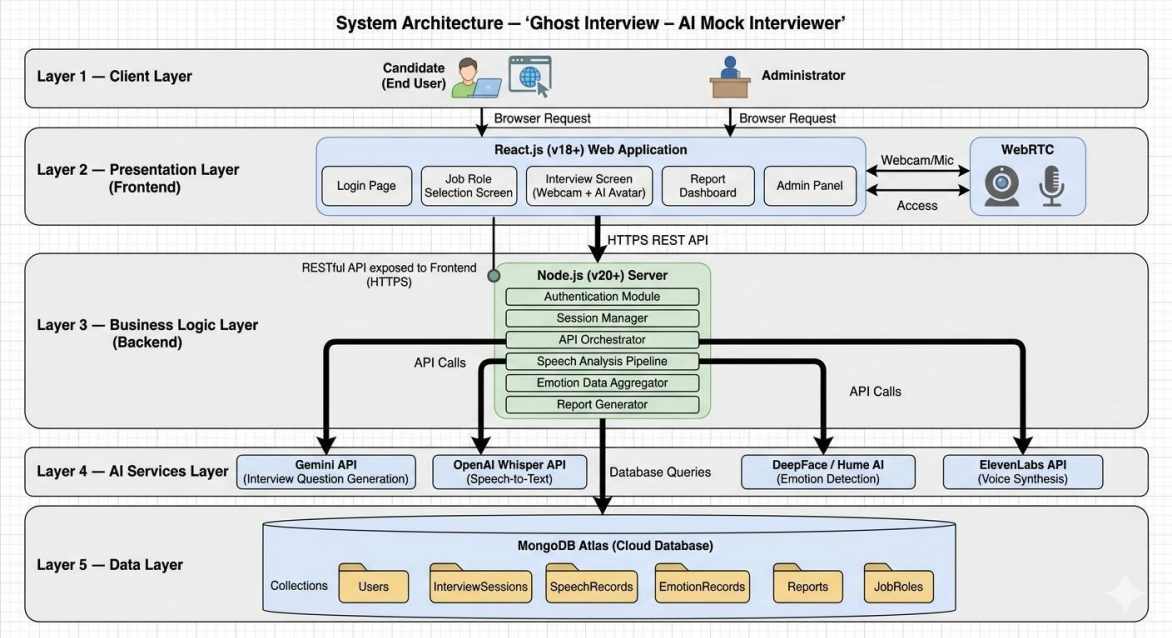
This document has presented the implemented modules of the Ghost Interview AI Mock Interviewer, together with their algorithms and screenshots captured from the live application (Chapter 5 / Implementation), and the software testing performed on those modules with both positive and negative scenarios (Chapter 6 / Software Testing). Every documented module is supported by a technical explanation, a step-by-step algorithm table, and evidence from the running system.

The features documented here represent approximately sixty percent of the overall project. The remaining work focuses on integrating the specified Hume/DeepFace server-side emotion provider, completing production HTTPS deployment with load and real-device camera/microphone acceptance testing, and extending analytics — all of which build directly on the working modules demonstrated and tested in this document.

Appendix A: System Design & Architecture Diagrams

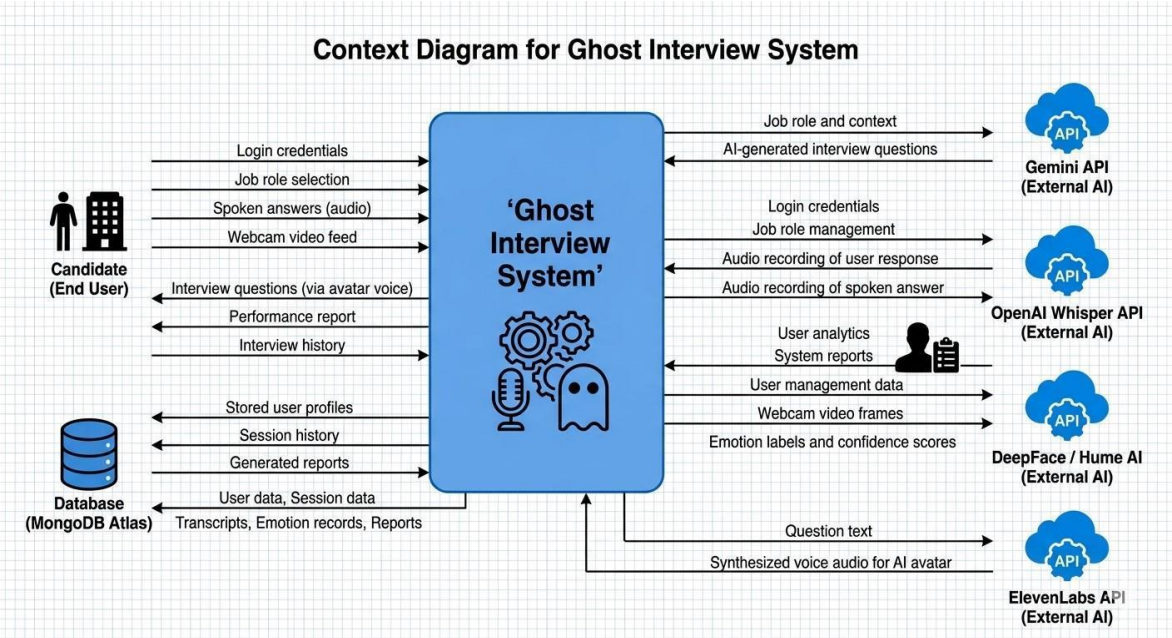
The diagrams below present the design artifacts that underpin the implemented modules described in Chapter 2.

System Architecture



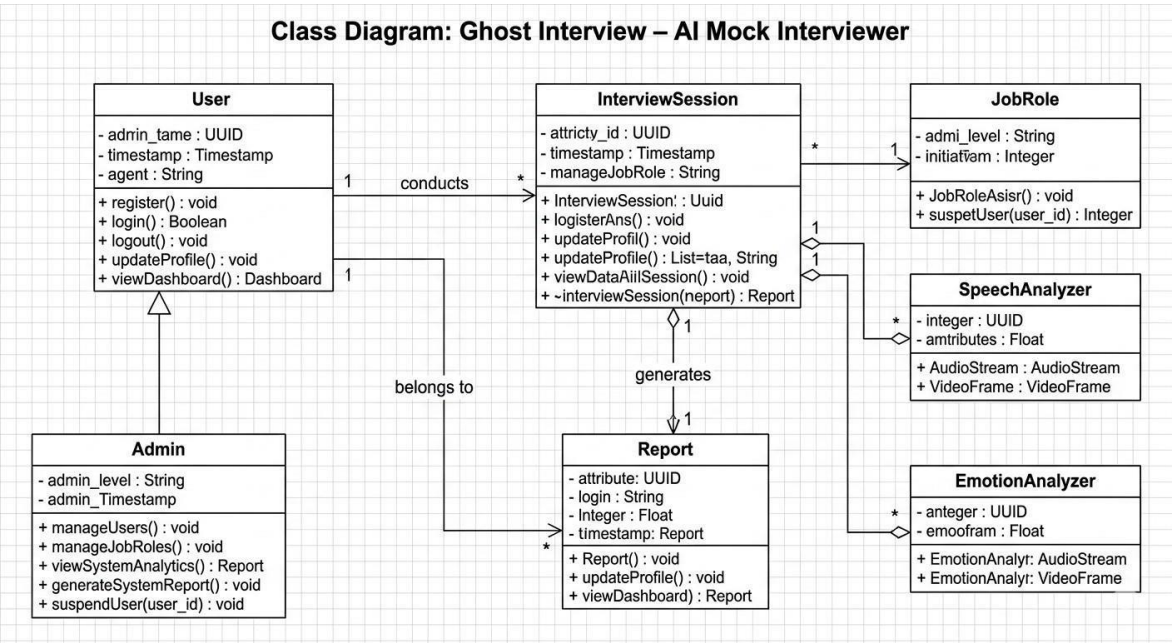
System architecture of the Ghost Interview platform (presentation, application, AI-service and data layers).

Context Diagram (Level-0 DFD)



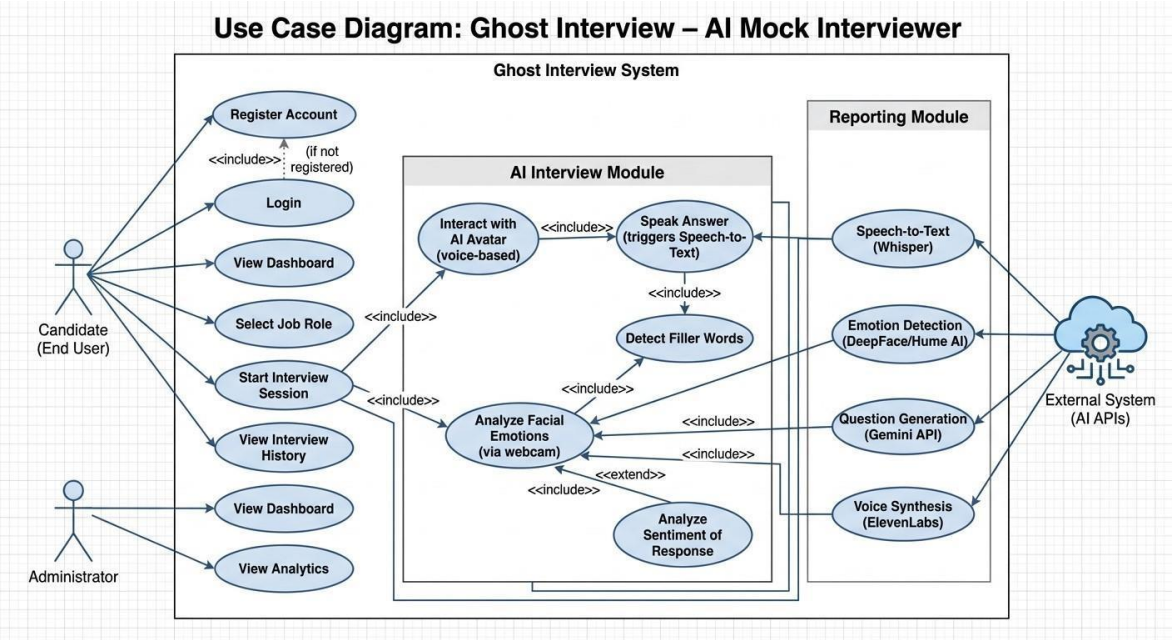
Context (Level-0) data-flow diagram showing the system boundary and external entities.

Class Diagram



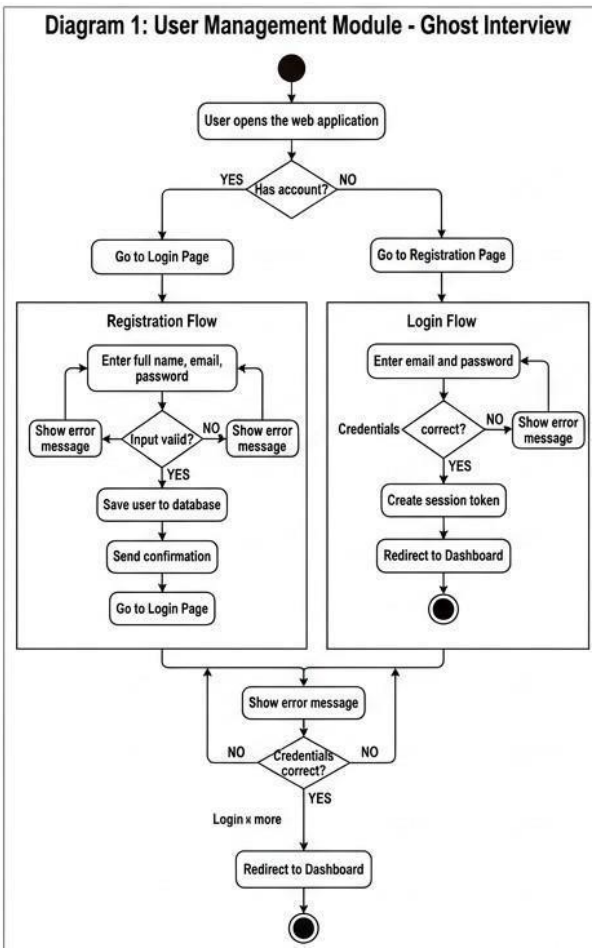
Class diagram of the core domain model: User, Admin, InterviewSession, JobRole, SpeechAnalyzer, EmotionAnalyzer and Report.

Use Case Diagram



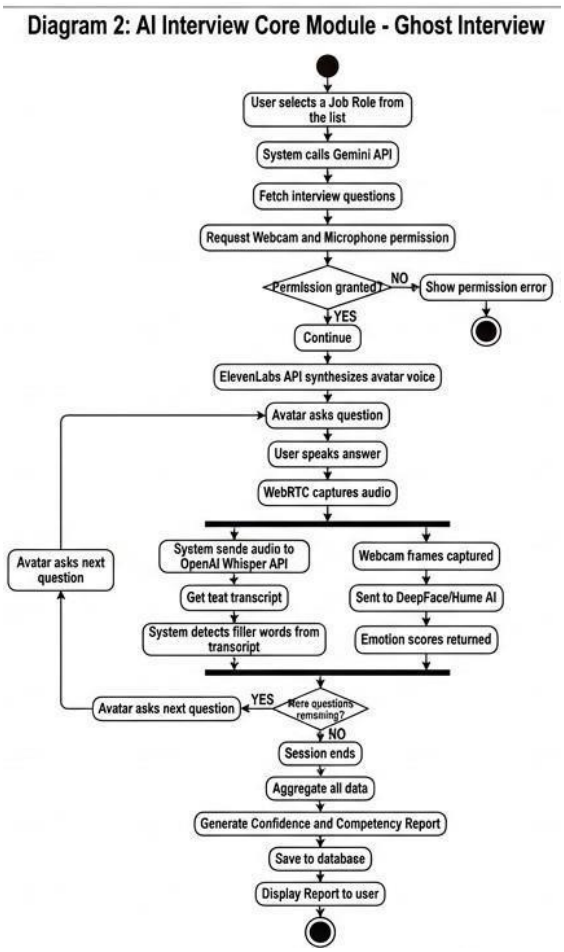
Use-case diagram showing candidate and administrator interactions with the system.

User Management Module Flowchart



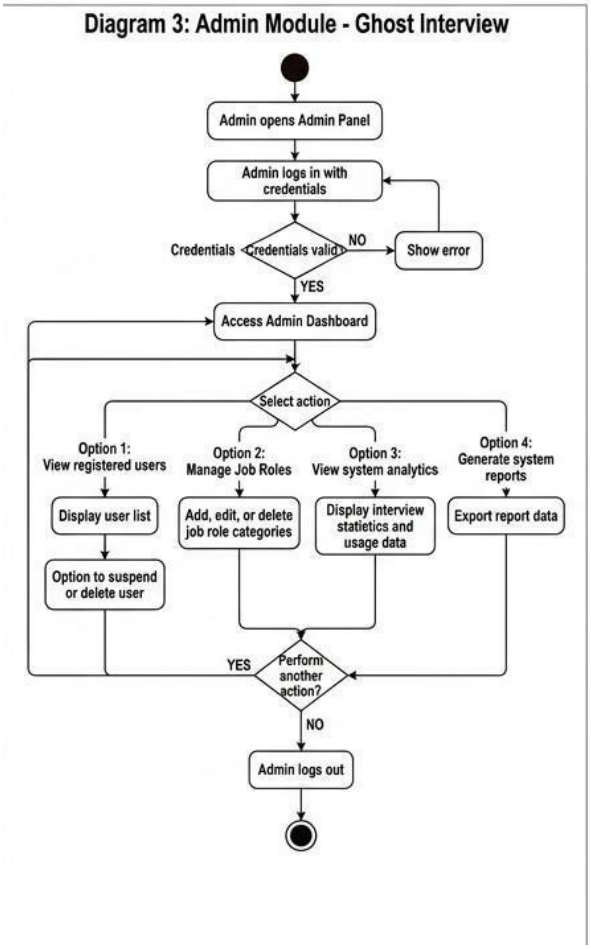
Activity flowchart of the User Management module (registration and login flows).

AI Interview Core Module Flowchart



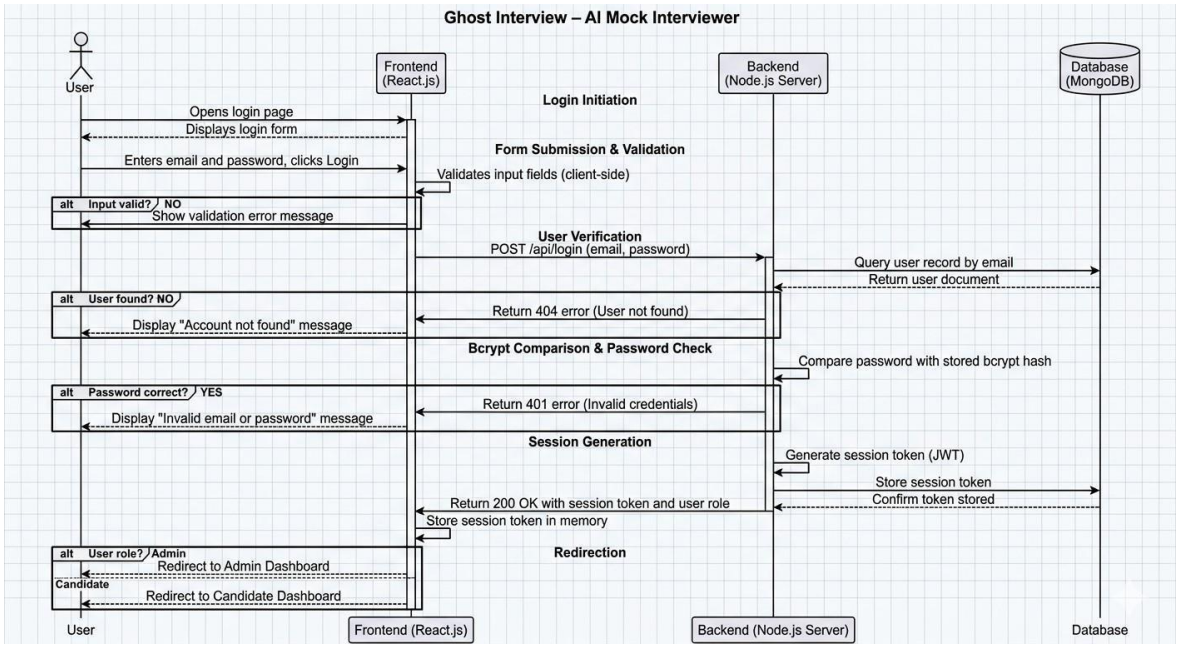
Activity flowchart of the AI Interview Core module (question generation, speech, emotion and scoring).

Admin Module Flowchart



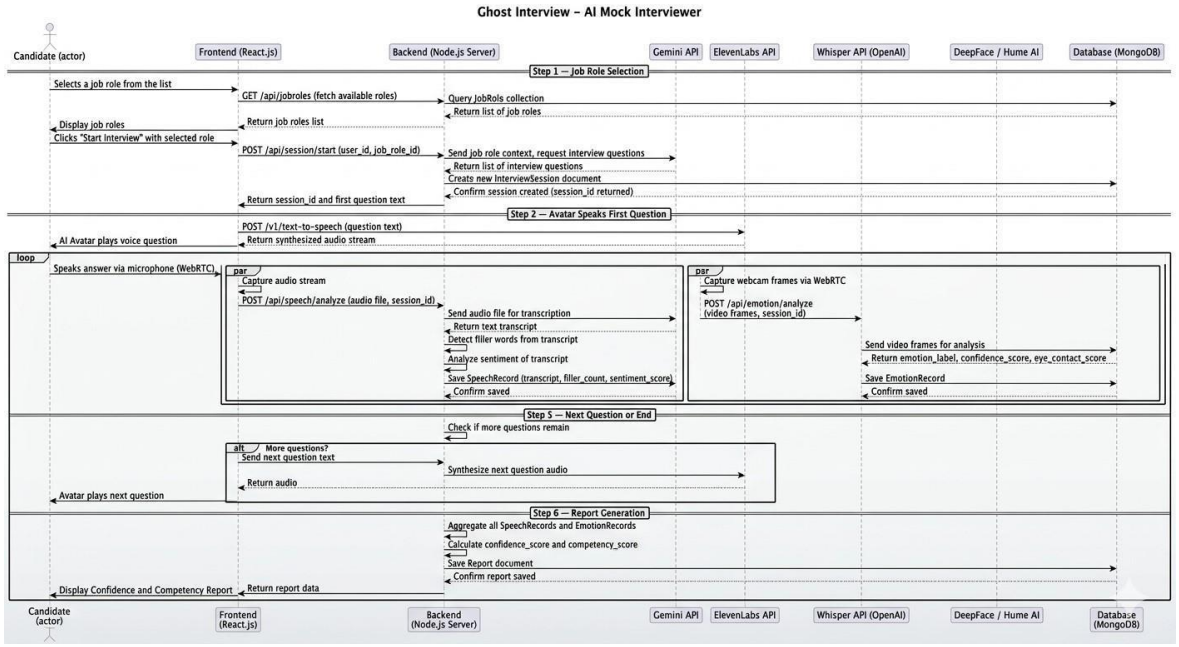
Activity flowchart of the Admin module (user, job-role, analytics and reporting actions).

Login Sequence Diagram



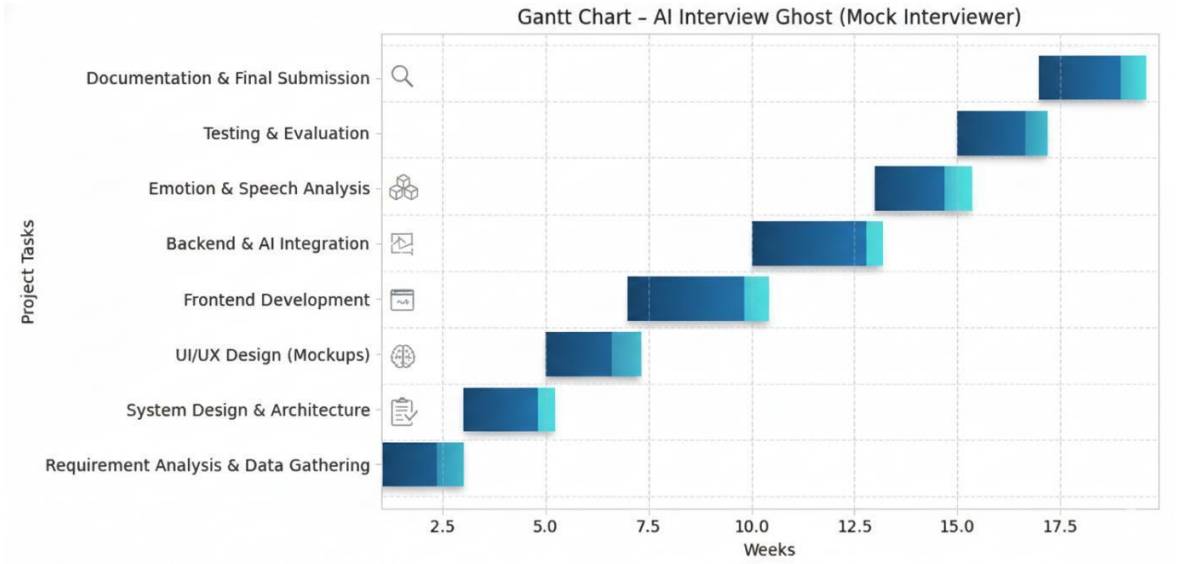
Sequence diagram for the candidate login and session-token generation flow.

Interview Session Sequence Diagram



Sequence diagram for a complete interview session across frontend, backend, AI services and database.

Project Timeline (Gantt Chart)



Project Gantt chart showing the development phases and milestones.